

Towards Detection of Software Supply Chain Attacks by Forensic Artifacts

Marc Ohm
ohm@cs.uni-bonn.de
University of Bonn
Bonn, Germany

Arnold Sykosch
sykosch@cs.uni-bonn.de
University of Bonn
Bonn, Germany
Fraunhofer FKIE

Michael Meier
mm@cs.uni-bonn.de
University of Bonn
Bonn, Germany
Fraunhofer FKIE

ABSTRACT

Third-party dependencies may introduce security risks to the software supply chain and hence yield harm to their dependent software. There are many known cases of *malicious* open source packages posing risks to developers and end users. However, while efforts are made to detect vulnerable open source packages, malicious packages are not yet considered explicitly. In order to tackle this problem we perform an exploratory case study on previously occurred attacks on the software supply chain with respect to observable artifacts created. Based on gained insights, we propose *Buildwatch*, a framework for dynamic analysis of software and its third-party dependencies. We noticed that malicious packages introduce a significant amount of new artifacts during installation when compared to benign versions of the same package. The paper presents a first analysis of observable artifacts of malicious packages as well as a possible mitigation strategy that might lead to more insight in long term.

CCS CONCEPTS

• **Security and privacy** → **Software and application security; Malware and its mitigation**; • **Software and its engineering** → Software creation and management.

KEYWORDS

Application Security, Malware, Supply Chain Attack, DevSecOps

ACM Reference Format:

Marc Ohm, Arnold Sykosch, and Michael Meier. 2020. Towards Detection of Software Supply Chain Attacks by Forensic Artifacts. In *The 15th International Conference on Availability, Reliability and Security (ARES 2020)*, August 25–28, 2020, Virtual Event, Ireland. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3407023.3409183>

1 INTRODUCTION

Supply chain attacks aim to inflict harm to a final target – company or user – by attacking less-secure components of the supply chain. The injection of malicious code into open source software packages constitutes one possible shaping of supply chain attacks. According

to the Internet Security Threat Report 2019 [11] by Symantec supply chain attacks increased over the course of the year 2018 and are very present on the threat landscape.

Recent attacks highlight the importance for a hardened open source software supply chain. For instance the npm package `event-stream` was hijacked through social engineering for malicious use. At that time 1,600 packages listed `event-stream` as dependency and the package was downloaded 1.5 million times a week on average. [6]

Malicious packages behave differently than benign packages. This behavior may yield a different set of forensic artifacts. We assume a malicious package maybe detected by the created artifacts during *installation*. **This work is focused on compromised packages, i.e., a package that an attacker gained publishing rights for, which he then uses to distribute a malicious version of an otherwise benign package.**

Our contribution is the initial analysis of forensic artifacts created by open source software packages during installation. The focus on compromised packages allows us to compare a malicious version of a package to a corresponding benign version that was previously released. Furthermore, continuous integration and continuous testing as part of DevOps and especially its augmentation DevSecOps have been widely adopted. A timely and automated process of integration and testing helps to improve a software’s quality in general as well as to reduce risks in regard to errors and security flaws [4]. Hence, we propose *Buildwatch*, a framework for dynamic analysis of software with a focus on third-party dependencies.

The remainder of this work is structured as follows. In Section 2 related work will be presented. Subsequently, required background information is presented in Section 3. The methodology and experimental design are detailed in Section 4. It is followed by the presentation of our results in Section 5 and the proposal of an analysis framework in Section 6. Section 7 concludes this paper and provides perspectives for future work.

2 RELATED WORK

Tschacher [12] created and published packages which leverage typosquatting on three popular package repositories. It was observed that package managers and their respective repositories are vulnerable to typosquatting attacks. Within two months 17,289 unique hosts had installed at least one of the prepared packages. Furthermore, nearly 50% of the package installations were executed with administrative rights. This work underlines the risks from third-party dependencies.

Carnogursky [2] scanned the Python package repository (PyPI) as part of his bachelor thesis to find anomalies by comparison of Abstract Syntax Trees (AST). The installation routine used by

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ARES 2020, August 25–28, 2020, Virtual Event, Ireland

© 2020 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-8833-7/20/08...\$15.00

<https://doi.org/10.1145/3407023.3409183>

malicious packages was deemed the main attack vector. While it is advised to sanitize package repositories like PyPI, we also encourage security checks within the local development pipeline.

Duan et al. [3] likewise leveraged AST in their vetting pipeline MALOSS which can be used to scan package repositories for suspicious packages. In addition, they employed dynamic analysis to monitor the packages' behaviors. Their approach is by far the most related to ours. We, however, are not interested in finding new suspicious packages on package repository level but in the local development environment.

Vaidya et al. [13] analyzed the package repositories npm and PyPI for security issues. It was found out that most packages on these platforms have multiple dependencies. A npm package has 86.55 dependencies on average while a python package on PyPI defines 7.33 dependent packages on average. Their conclusion indicates that automated detection of malicious packages at scale might not be feasible. However, tools for developers that assist decision-making are proposed.

Bass et al. [1] focused on securing a deployment pipeline against supply chain attacks. They proposed several hardening recommendations to mitigate the impact of compromise on different pipeline stages. While this work focuses on the hardening of the software against attacks, we would like to raise awareness and make third-party dependencies' impacts visible to the developer. This is where we want to catch up by making introduced changes by third-party dependencies visible to the developer.

Ohm et al. [8] manually analyzed malicious open source packages from npm, PyPI, and RubyGems published between 2015 and 2019. According to their analysis, most malicious packages expose their behavior during installation. Their dataset is freely available at GitHub and will be used as source of malicious packages for our experiments.

In contrast to related work, we conduct an exploratory case study on previously occurred attacks on the software supply chain with respect to forensic artifacts created. Furthermore, we propose an analysis framework which may be adopted into the DevSecOps pipeline of developers.

3 BACKGROUND

Nowadays, software is neither developed by one person nor a monolithic construct. Besides the original code, software comprises several third-party packages used for common functionality.

Typically, software is developed using a version control system like Git. Automation tools like GitLab Runner, Jenkins, Travis, etc. may be used to freely define jobs that automatically test, integrate, and publish the software. Required dependencies are commonly listed in the software package's definition, e.g. `package.json` for Node.js, `setup.py` and `requirements.txt` for Python. During the build – or installation – of the software, required dependencies are automatically pulled from the corresponding package repository like npm (Node.js) or PyPI (Python).

It is considered good practice to use version pinning, i.e. state the exact version the software package depends on. This yields reproducible results across environments and prevents from breaking changes introduced by listed dependencies. Moreover, Semantic Versioning [9] allows distinguishing between *patch*, *minor*, and

Package	Affected	Reference	Origin
mariadb	2.13.0	2.2.0, 2.1.5, 2.1.4	npm
opencv.js	1.0.1	1.0.0, 1.1.0, 1.1.1, 1.1.2	npm
kraken-api	0.1.8	0.1.7, 0.1.6, 0.1.5	npm
rpc-websocket	0.7.7	0.7.6, 0.7.5, 0.7.4	npm
eslint-scope	3.7.2	3.7.1, 3.7.0	npm
eslint-config-eslint	5.0.2	5.0.1, 5.0.0, 4.0.0	npm

Table 1: Malicious open source packages considered for analysis.

major releases which helps to estimate the occurrence of breaking changes. Thus, both npm¹ and pip² support acceptable version ranges, e.g. allow bug fixes and security patches while preventing installation of dependencies with breaking changes.

However, if an attacker gains rights to publish a new version of a package the automated update of dependencies becomes a powerful attack vector. The sheer amount of dependencies makes manual review infeasible. However, using exact version pinning makes the software prone to vulnerabilities through missed security patches of dependencies.

Forensic artifacts may be expressed in different formats. For the extent of this paper we express forensic artifacts in form of *STIX Cyber Observable Objects* (short: *observables*) [7]. The two terms will be used interchangeably throughout the remainder of the paper.

4 METHODOLOGY

This work aims at the extraction and analysis of forensic artifacts from open source software packages which may be used as third-party dependency. An observable in this case refers to an artifact, like *files created* or *hosts connected*, occurring while the software package is installed. We focus on introduced changes in the set of forensic artifacts between benign and malicious versions of the same package. Thus, we propose the following hypothesis:

H1 Malicious packages introduce a noticeable change in exposed forensic artifacts.

Candidates for manual inspection are packages that were compromised in the past and are (1) publicly reported as malicious, (2) infected an existing package, and (3) show malicious behavior during installation. In order to establish a baseline, benign versions have to be available as well. Manual research [8] yielded this combination of characteristics in only seven instances. Although, only limited inference may be drawn based on observation on such a small sample size it provides a very illustrative and conclusive proof of concept.

The list of considered packages can be found in Table 1. In case of `opencv.js` we used three subsequent, consecutive versions rather than the previous version as there is only one benign previous version. For `eslint-scope` only two previous versions are available.

We did not analyze `event-stream` as it is targeted against a crypto wallet and only shows maliciousness if enough crypto currency is in the wallet.

¹<https://docs.npmjs.com/about-semantic-versioning>

²https://pip.pypa.io/en/stable/reference/pip_install/#requirement-specifiers



Figure 1: Transformation Example: Each Python function invokes a corresponding system call. Used system calls, flags, and return values are used to infer an observable description.

Collected packages are installed in a sandboxed environment where resulting forensic artifacts may be recorded safely. For the analysis we leverage Cuckoo which is a powerful, open-source sandbox for “automating analysis of suspicious files” [5]. We use a Linux-based virtual machine to perform dynamic analysis by execution and monitoring of said malicious packages. Monitoring is realized by Cuckoo’s implementation of `strace` [10] to capture all system calls made by the executed file, i.e. kernel services requested by processes.

In a post-processing step, forensic artifacts are extracted from collected system calls. Forensic artifacts like created files or opened network connections can be inferred from system calls like `open`³ and `socket`⁴, respectively. A mapping of system calls to observables can be found in Table 2. Relying on system calls allows generalization of our approach to other programming languages.

Figure 1 provides a straight forward example for the transformation from a script written in Python, into a list of system calls, to a definition of an observable.

Similar to this example, observables are extracted from results of dynamic analyses. However, Cuckoo, `git`, and `npm` create randomly named directories and filename suffixes on each execution. Hence, it was necessary to homogenize file paths for better comparability. For instance the filepath `/tmp5Wut0v/node_modules/underscore/package.json.3993641503` is homogenized to `./node_modules/underscore/package.json`. Based on that, we are able to compare the sets of forensic artifacts produced by malicious and corresponding benign versions of a package.

The set of newly introduced changes after the version increment is based on the difference of the sets as follows.

$$A - B = \{x \mid x \in A \text{ and } x \notin B\} \quad (1)$$

Thus, the difference consists of elements present in set A but not in set B . Where A represents the incremented – compromised – version and B is the previous (reference) version.

³<https://linux.die.net/man/2/open>

⁴<https://linux.die.net/man/2/socket>

System Call	Flags	Scope	Action
open, openat	O_WRONLY, O_RDWR, O_APPEND, O_CREAT	file	create
creat, link, symlink, rename		file	create
fopen	w, w+, a, a+	file	create
open, openat	O_WRONLY, O_RDWR	file	read
fopen	r, r+	file	read
unlink, unlinkat		file	remove
mkdir		directory	create
rmdir		directory	remove
execve		process	create
connect	AF_INET, AF_INET6	IP	connect

Table 2: Mapping of system calls and corresponding flags to observables.

5 RESULTS

All versions of the packages listed in Table 1 were run with Cuckoo on an Ubuntu 18.04.3 (VirtualBox) guest system. The virtual machine was provided with internet access so that required packages, as listed in the package’s definition, could be obtained from `npm`, thus allowing successful installation. System calls and respectively observables were extracted to the methodology described in Section 4.

In Figure 2, the difference in the set of observables between two versions is displayed. The compromised version of the package is denoted as t_0 . For `rpc-websocket` this would be $0.7.4 \xrightarrow{t_{-2}} 0.7.5 \xrightarrow{t_{-1}} 0.7.6 \xrightarrow{t_0} 0.7.7$.

For all packages except of `rpc-websocket` a significant amount of changes is introduced with the compromised version at t_0 . A benign version increment introduces 6.45 ($min = 4, max = 16, \sigma = 3.96, \tilde{x} = 4$) new observables on average. A malicious version increment however, led to 90.67 ($min = 7, max = 435, \sigma = 155.15, \tilde{x} = 13$) new observables on average. This is an increase of 1,305.74% on average. The fairly big difference on average is a result of the package `mariaadb`’s unusual high amount of new observables exposed with the compromised version (c.f. below). However, even the median of 4 new observables for benign and respectively 13 for malicious version increments (+225%) allow us to verify our hypothesis H1.

5.1 Per Package Analysis

The following paragraphs will briefly discuss the observables of each considered package. It has to be pointed out that the change in

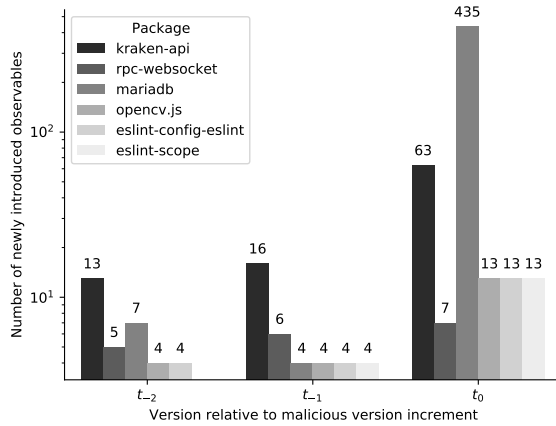


Figure 2: New observables introduced in benign and malicious version increment per considered package. The malicious version increment is denoted as t_0 .

forensic artifacts resulting from an adjustment to the version number may be quite extensive due to the mechanics of the packaging system. These changes also vary by the size of the package itself.

kraken-api. All 13 new observables exposed by *kraken-api* in t_{-2} are a direct result of the changes related to the version number increment. Given the fact, that t_{-2} it is a *patch* release, this observation is expected. In t_{-1} , 16 new observables occurred. 15 of them are caused by the version number adjustment. However, a new file (*.npmignore*) was added to the package.

In the difference t_0 63 new observables are exposed. This is due to the newly introduced dependency *daemon* as well as one new file: *lint.js*. Furthermore, the file was executed in a new process (`sh -c $NODE lint.js`) which established a new network connection to 95.213.253.26:443.

mariadb. Similar to *kraken-api* the changes t_{-2} reflect only a *patch* release. However, the changes in t_{-1} are not extensive either. They mainly contain new observables as a result to the version number adjustment. But in t_0 several new dependencies and processes were added to the package. This results in over 400 new observables compared to only seven and four new observables on the differences between the reference versions (ref. Figure 2). However, the high amount of new observables on its own does not indicate maliciousness and might stem from benignly introduced dependency. It has to be pointed out, that the difference was calculated for the version increment from 2.2.0 to 2.13.0, spanning multiple *minor* releases, thus more changes are to be expected.

Apart from new dependencies, a new file (*package-setup.js*) was introduced which is executed in a new process (`sh -c node package-setup.js`). A manual analysis of that file revealed a small script that sends environment variables – possibly containing access tokens – via a POST request to the domain *npm.hacktask.net*. However, the host appeared to be unreachable at the time of this study. Hence, no further observables like the network connection could be observed.

opencv.js. For *opencv.js*, benign differences include four observables caused by version number adjustment only. The malicious version introduced a new file (*package-setup.js*) which is executed by a new process (`sh -c node package-setup.js`). Manual analysis revealed the same code as used in *mariadb*.

rpc-websocket. In the case of *rpc-websocket* the number of new observables is not significantly higher than the number of changes from benign updates. However, looking at the observables a new, suspicious process (`sh -c nohup /bin/sh -c 'nc 185.61.148.117 443 | /bin/sh' >/dev/null 2>&1 &\r> jshint *.js`) is added. The resulting network connection could not be observed due to the host being unreachable and hence no observable connection was established.

eslint-scope & *eslint-config-eslint*. Again, most of the new observables in benign version increments are results of the new version number or due to npm’s loadbalancing techniques (c.f. below).

In t_0 of both *eslint-scope* and *eslint-config-eslint*, several new Observables related to a new file (*lib/build.js*) pop up. Similar to previous mentioned packages, the new file is executed in a new process which is also listed in the difference. Furthermore, the connection to an IP address (104.23.98.190), which belongs to *pastebin.com*, is visible. *Pastebin* allows to publicly store text snippets which may be used as actual malicious code in a software supply chain attack. However, the corresponding *pastebin* site was taken down already.

Overall it is noticeable that malicious versions tend to introduce an unusual amount of new observables. Most often a new file containing malicious code is added and subsequently executed in a new process. Nonetheless, suspicious observables may be detected even if the number of new observables is small. Probably even easier as they stick out even more.

5.2 Per Observables Analysis

In Figure 3 introduced observables are broken down into categories as listed in Table 2. Again it is noticeable that malicious version increments introduce more new observables than benign version increments. This is especially true for observables related to files.

The significant increase is most probably due to *kraken-api* and *mariadb* adding new dependencies which lead to more file-related operations. There are no removed files on average for benign packages but 38 for malicious packages. This is a result of the installation of *mariadb* which failed due to the unreachability of the remote host. Thus, npm performed a rollback of installed files. Nonetheless, an increased amount of connected hosts and started processes is visible.

If Semantic Versioning (c.f. Section 3) was employed by the considered packages, then for five out of six analyzed packages maliciousness was introduced on the patch-level. In contrast to that, the malicious code in *mariadb* was added on minor-level. However, the amount of changes in previous version increments – patch- and minor-level – is comparatively low.

Introducing malicious code as patch-level change has advantages. Often version pinning is used that allows automatic upgrades of patch and minor versions to get security and bug fixes. However,

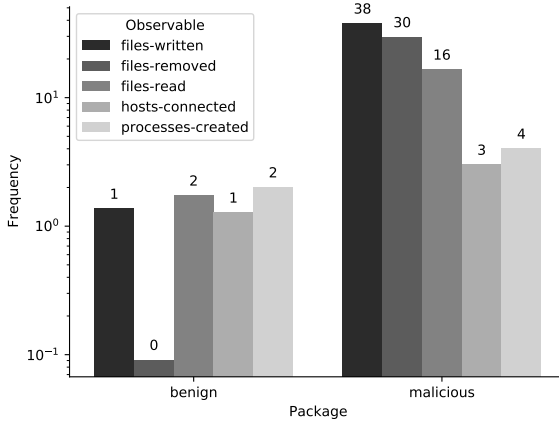


Figure 3: Average of newly introduced observables by type for benign and malicious version increment.

this allows the attacker to infect a system if the malicious code was obfuscated as patch-level change.

Packages from npm depend on nearly 90 other packages on average [13]. Thus, it is not feasible to analyze them by hand at scale. However, if one of these packages contains malicious code it will propagate into all dependent packages.

Since continuous integration has become common practice, it is a good idea to adopt our approach to that environment. Thus, we propose *Buildwatch*, a dynamic code analysis plugin for CI environments.

6 BUILDWATCH

Typically, code resides in a version control system like Git. Automation tools like GitLab Runner, Jenkins, Travis, etc. may be used to freely define CI jobs. New changes are then automatically tested, integrated, and published by defined CI jobs. This is where Buildwatch hooks in.

Note that Buildwatch is not meant to analyze packages for maliciousness but rather raise awareness to changes of system activities caused by dependent packages that are not under ones control.

6.1 Architecture

As displayed in Figure 4, an additional CI job conducts the dynamic analyses of the install-procedure during which the dependencies are downloaded and installed as well. In our current implementation the job bundles the whole repository together with introduction on how to build and run the software (`.buildwatch.sh`). That bundle is submitted to an instance of Cuckoo, possibly running on another host. During submission additional information like the required build environment may be passed to Cuckoo. Cuckoo will boot up the appropriate sandbox and start the dynamic analysis.

Our Buildwatch-extension for Cuckoo will consume these instructions and build/run the code accordingly. This includes the installation of third-party dependencies as listed in the package's definition e.g. `package.json`. During installation, Cuckoo will record

all system calls made. Buildwatch extracts observables from these system calls, remove blacklisted observables (i.e. noise), and calculate the difference to previous behavior.

That way we are able to utilize the capabilities of dynamic analysis on a very early stage of a software's life cycle. If one of the included dependencies becomes malicious it may be detected before it can cause harm to the build systems or downstream users.

6.2 Feedback to Developers

At last, results are fed back to the CI and displayed to the developer. The developer may now decide whether suspicious observables occurred.

Listing 1 shows the output for `rpc-websocket` version 0.7.7 against version 0.7.6. It is notable that no new files were written, as depicted on Line 2. Line 4 indicates a file removal that was not present in the last version. However, this might be considered noise. The new observable for the read file `rpc-websocket-0.7.7.tgz` on Line 7 originates in the version upgrade and is thus benign.

Line 10 and 11 display two previously unseen connected hosts. However, the IP address on Line 10 belongs to npm (cloudflare)⁵. The IP address on Line 11 belongs to GitHub⁶. Due to npm's and GitHub's load balancing techniques their IP addresses differ often. Thus, it may be a good idea to exclude them from the reports.

Starting from Line 13, newly created processes are listed. Processes on Line 14 and 15 are related to node and npm and belong to the package installation. They illustrate the changes that accompany version number adjustment.

The process listed on Line 16 opens a network connection (`nc`)⁷ to 185.61.148.117:443, aside from the package management system and therefore has to be treated as suspicious, at least. The connection is used as reverse shell (`/bin/sh`) which is pushed to

⁵<https://www.cloudflare.com/ips/>

⁶<https://api.github.com/meta>

⁷<https://linux.die.net/man/1/nc>

```

1 {
2   "files_written": [],
3   "files_removed": [
4     "./node_modules/.staging"
5   ],
6   "files_read": [
7     "./rpc-websocket-0.7.7.tgz"
8   ],
9   "hosts_connected": [
10    "104.16.25.35:443",
11    "140.82.118.4:9418"
12  ],
13  "processes_created": [
14    "node /usr/local/bin/npm install rpc-
15      websocket-0.7.7.tgz",
16    "npm install rpc-websocket-0.7.7.tgz",
17    "sh -c nohup /bin/sh -c 'nc 185.61.148.117
18      443 | /bin/sh' >/dev/null 2>&1 &#\\r>
19      jshint *.js"
20  ]
21 }
```

Listing 1: Differences between `rpc-websocket` 0.7.6 and 0.7.7.

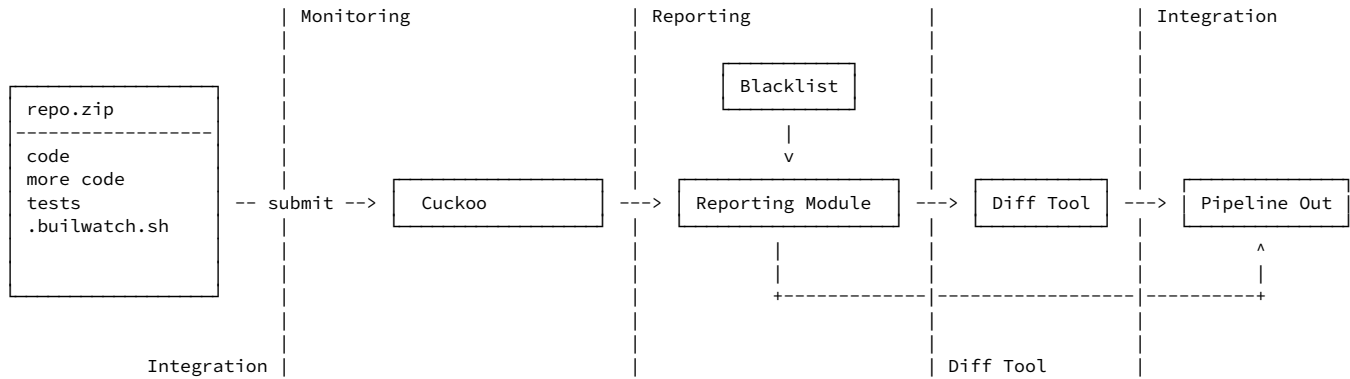


Figure 4: Architecture of Buildwatch, a CI extension for dynamic analysis.

the background (nohup⁸) for the statement to return and the installation to proceed.

Note that there is no automated vetting at this point. Until now, it is up to the developer to decide if an observable is benign, suspicious, or even malicious. However, an automated assessment of new observables is definitely up for future work.

7 CONCLUSION

We analyzed the changes from one version to another of compromised npm packages. We focus on the releases that introduced malicious behavior to these packages and compared these changes to those of the last benign release as a baseline. Our analysis is based on forensic artifacts as a means to comprehend host change opposed to raw system call logs.

Although the inspected sample size is quite limited, we were able to show that malicious behavior may be detected based on these artifacts. We observed an increased number of observables generated when a package becomes malicious. Most often newly introduced observables are related to operations on files. Additionally, previous unseen processes are created which are used to run the malicious additions.

These observations lead to the proposal of Buildwatch, a dynamic software dependency monitoring system for a continuous integration pipeline. Buildwatch is able to assist the developer in the assessment whether a dependency update introduced malicious behavior.

However, except for mariadb, these versions are all patch-level version increments. This is most probably the explanation for the low number of newly introduced observables between benign version number increments. Changes on patch-level typically do not introduce new features and hence seldom result in new observables. However, due to the age of the malicious packages analyzed, a reduced set of malicious observables is to be expected as command and control hosts were already taken down and hence no further artifacts were able to be observed. Thus, even more malicious observables might occur in the wild. For now, we analyzed packages that trigger their malicious behavior during installation. However, our approach is transferable to packages that perform malicious actions during runtime in general.

⁸<https://linux.die.net/man/1/nohup>

For future work we would like to perform long term analysis of behavior of observables over a software's lifetime. Furthermore, it is desirable for the system to adapt a baseline of benign observables created by version increments. This way newly introduced observables may be checked against their statistical probability to occur. Unusual observables could be highlighted, reducing the developers work load.

ACKNOWLEDGMENTS

This work is funded under the SPARTA project, which has received funding from the European Union's Horizon 2020 research and innovation programme under grant agreement No 830892.

REFERENCES

- [1] Len Bass, Ralph Holz, Paul Rimba, An Binh Tran, and Liming Zhu. 2015. Securing a deployment pipeline. In *2015 IEEE/ACM 3rd International Workshop on Release Engineering*. IEEE, 4–7.
- [2] Martin Čarnogurský. 2019. *Attacks on Package Managers*. Master's thesis. Masaryk University, Faculty of Informatics.
- [3] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. 2020. Measuring and preventing supply chain attacks on package managers. *arXiv preprint arXiv:2002.01139* (2020).
- [4] Paul M Duvall, Steve Matyas, and Andrew Glover. 2007. *Continuous integration: improving software quality and reducing risk*. Pearson Education.
- [5] Stichting Cuckoo Foundation. 2019. Cuckoo Sandbox - Automated Malware Analysis. Retrieved February 21st, 2020 from <https://cuckoosandbox.org>
- [6] Th. Hunter II. 2018. Compromised npm Package: event-stream. Retrieved February 25th, 2020 from <https://medium.com/intrinsic/compromised-npm-package-event-stream-d47d08605502>
- [7] OASIS. 2017. STIXTM Version 2.0. Part 3: Cyber Observable Core Concepts. <https://docs.oasis-open.org/cti/stix/v2.0/cs01/part3-cyber-observable-core/stix-v2.0-cs01-part3-cyber-observable-core.html>
- [8] Marc Ohm, Henrik Plate, Arnold Sykosh, and Michael Meier. 2020. Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks. In *17th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer.
- [9] Tom Preston-Werner. 2020. Semantic Versioning 2.0.0. Retrieved April 15th, 2020 from <https://semver.org/>
- [10] strace. 2020. strace - the linux syscall tracer. Retrieved February 21st, 2020 from <https://strace.io/>
- [11] Symantec. 2019. Internet Security Threat Report. Retrieved February 25th, 2020 from <https://www.symantec.com/content/dam/symantec/docs/reports/istr-24-2019-en.pdf>
- [12] Nikolai Philipp Tschacher. 2016. *Typosquatting in programming language package managers*. Master's thesis. Universität Hamburg, Fachbereich Informatik.
- [13] Ruturaj K Vaidya, Lorenzo De Carli, Drew Davidson, and Vaibhav Rastogi. 2019. Security issues in language-based software ecosystems. *arXiv preprint arXiv:1903.02613* (2019).